

INSTITUCION UNIVERSITARIA PASCUAL BRAVO

Departamento de Sistemas Digitales

Trabajo de consulta

SolarDB Pascual: gobernanza de datos, automatizacion ETL e IoT sobre un repositorio relacional

Integrantes:

Camilo Velasquez Botero

Deivy Estiben Gomez Rivera

Curso: Bases de Datos I (SD1006)

Grupo: 811

Semestre: 2026-II

Docente: Ramiro Grisales Montoya

Fecha: Octubre de 2026

Repositorio: <https://github.com/CamiloVelasquezBotero/solardb-velasquez-botero>

Trabajo realizado en pareja.

Parte A. Consulta

Las respuestas estan redactadas con palabras propias y aplicadas al caso SolarDB. Las fuentes se citan entre corchetes y se listan en la seccion de referencias.

Bloque 1. Gobernanza de datos (data governance)

G1. Data governance frente a data management y data quality

Segun DAMA-DMBOK [1], data governance es el ejercicio de autoridad y control sobre la gestion de los activos de datos: define politicas, estandares, roles y quien decide que. Data management es el conjunto mas amplio de actividades que ejecutan el trabajo con los datos (arquitectura, modelado, almacenamiento, seguridad, integracion, metadatos); la gobernanza es una de sus areas y la que orienta a las demas. Data quality es otra de esas areas y se ocupa de medir y mejorar si los datos sirven para su uso. En SolarDB, la gobernanza decide que la telemetria debe ser trazable y que solo el rol de ingesta escribe en lectura; la gestion implementa tablas, cargas y respaldos; la calidad verifica que p_ac no sea negativa ni existan lecturas repetidas. Sin gobernanza, las otras dos no tendrian reglas ni responsables.

G2. Roles data owner, data steward y data custodian

El data owner tiene la autoridad y la responsabilidad final sobre un dominio de datos: aprueba su uso, sus accesos y sus reglas de calidad. El data steward se encarga del dia a dia: mantiene definiciones, unidades y reglas, y atiende los problemas del dato. El data custodian es el rol tecnico que almacena, protege y respalda los datos segun lo decidido por el owner [1]. Para SolarDB propongo: owner, el responsable institucional de la planta solar, porque responde por sus datos ante la universidad; steward, quien mantiene el modelo y el diccionario, porque conoce el significado, las unidades y los rangos validos; custodian, quien administra PostgreSQL (rol solar_admin), porque ejecuta permisos, respaldos y cargas. En este trabajo, Camilo Velasquez asume el rol de steward y Deivy Gomez el de custodian, y el owner sigue siendo institucional.

G3. Metadatos, catalogo de datos y brechas del diccionario

Los metadatos son datos que describen a otros datos: su significado, estructura, origen, calidad, responsable y uso [1]. Un catalogo de datos es el inventario consultable de esos metadatos, que permite encontrar, entender y confiar en cada activo. Un diccionario basico como el del Taller 1 suele describir tabla, columna, tipo, nulabilidad, llaves y descripcion; para funcionar como catalogo gobernado le faltan las columnas de la tabla siguiente, que permiten saber quien responde por cada campo, de donde viene y cuando se reviso por ultima vez.

Columna a agregar	Ejemplo en SolarDB
responsable (owner / steward)	lectura.p_ac: steward = responsable del modelo
origen del dato	Inversor 1 via gateway MQTT, topic .../inv01/p_ac
regla de calidad	CHECK (p_ac >= 0); NOT NULL
fecha de ultima actualizacion	2026-10-05 (fecha de la ultima revision del campo)
unidad de medida	kW
clasificacion de sensibilidad	Publico / interno / personal

G4. Dimensiones de calidad y restricciones de PostgreSQL

La calidad de datos se evalua por dimensiones [1]: completitud (estan todos los datos requeridos), validez (cumplen formato y rango), unicidad (sin duplicados), consistencia (coinciden entre tablas y reglas), oportunidad (llegan a tiempo para su uso) y exactitud (representan la realidad medida). En SolarDB, la completitud se traduce en NOT NULL sobre dispositivo_id y ts; la validez, en CHECK sobre rangos fisicos; la unicidad, en la clave primaria que impide dos lecturas del mismo equipo en el mismo instante; la consistencia, en una FOREIGN KEY de lectura hacia dispositivo, que evita lecturas de equipos inexistentes. La oportunidad y la exactitud no se imponen solo con restricciones: se monitorean comparando ts con cargado_en y calibrando sensores.

```
-- Completitud : NOT NULL          -- Validez : CHECK
ALTER TABLE lectura_demo ALTER COLUMN ts SET NOT NULL;
ALTER TABLE lectura_demo ADD CONSTRAINT ck_p_ac CHECK (p_ac >= 0);
-- Unicidad : PRIMARY KEY / UNIQUE
ALTER TABLE lectura_demo ADD CONSTRAINT pk_lectura PRIMARY KEY (dispositivo_id, ts);
-- Consistencia : FOREIGN KEY (tabla dispositivo del modelo oficial)
ALTER TABLE lectura ADD CONSTRAINT fk_lectura_disp
FOREIGN KEY (dispositivo_id) REFERENCES dispositivo (dispositivo_id);
```

G5. Minimo privilegio y roles de SolarDB

El principio de minimo privilegio [12] establece que cada usuario o proceso debe tener solo los permisos indispensables para su funcion. Reduce el dano de errores, malware o credenciales robadas. En PostgreSQL se aplica creando roles con CREATE ROLE, quitando permisos por defecto con REVOKE y entregando solo lo necesario con GRANT [3]. Para SolarDB defino tres roles: solar_ingesta, usado por el proceso ETL, que escribe en staging, lectura y bitacora; solar_lector, para consulta, que solo lee lectura y bitacora y no ve el staging con mensajes crudos; y solar_admin, para el custodio, con control total para mantenimiento. Ningun rol de aplicacion es superusuario. El codigo completo esta en sql/03_roles.sql.

Tabla	solar_ingesta	solar_lector	solar_admin
stg_lectura_raw	SELECT, INSERT, TRUNCATE	sin acceso	ALL
lectura_demo	SELECT, INSERT	SELECT	ALL
etl_log	SELECT, INSERT, UPDATE	SELECT	ALL

```
CREATE ROLE solar_lector LOGIN PASSWORD :pwd_lector;      -- clave por variable de psql
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
GRANT SELECT ON lectura_demo, etl_log TO solar_lector;
GRANT SELECT, INSERT ON lectura_demo TO solar_ingesta;
```

G6. Ley 1581 de 2012 y postura etica

La Ley 1581 de 2012 [2] protege los datos personales, es decir, cualquier informacion vinculada o que pueda asociarse a una persona natural determinada o determinable. La telemetria de la planta (potencia, irradiancia, temperatura de modulo) describe equipos y condiciones fisicas, no personas; por tanto no es dato personal. Si SolarDB registrara a los operadores que atienden alarmas (nombre, documento, hora y accion), esos registros si lo serian y habria que cumplir los principios de finalidad, transparencia, seguridad y circulacion restringida, pedir autorizacion previa, expresa e informada, respetar los derechos de conocer, actualizar, rectificar y suprimir, y evaluar la inscripcion de la base ante la SIC [13]. Postura etica: recolectar solo lo necesario, usar un identificador seudonimo, restringir el acceso por rol y fijar tiempo de retencion.

Bloque 2. Automatizacion de procesos ETL

E1. ETL frente a ELT

ETL (extract, transform, load) transforma los datos en un motor intermedio antes de cargarlos al destino; ELT (extract, load, transform) carga primero el dato crudo en el destino y lo transforma alli con la capacidad de la base [15]. SolarDB se parece mas a ELT: el mensaje del gateway llega intacto a stg_lectura_raw.payload (JSONB) y despues una sentencia INSERT ... SELECT extrae los atributos con ->>, los convierte de tipo y los inserta en lectura_demo; la transformacion ocurre dentro de PostgreSQL. Esto conserva el dato original para auditar o reprocesar si cambian las reglas. El unico paso previo es una validacion ligera de que cada linea sea JSON valido con device_id y ts, por lo que en rigor es un ELT con una pequena validacion antes de la carga.

E2. Carga completa, carga incremental e idempotencia

En una carga completa (full load) se borra y se vuelve a cargar todo el conjunto; es simple pero costosa cuando la serie de tiempo crece. En una carga incremental solo se procesan los datos nuevos desde la ultima ejecucion, por ejemplo los de ts mayor al ultimo cargado. Una carga es idempotente cuando ejecutarla una o varias veces con la misma entrada deja el mismo resultado final [14]. Con INSERT ... ON CONFLICT (dispositivo_id, ts) DO NOTHING [3], la clave primaria de lectura identifica cada medicion; si ya existe, PostgreSQL la omite sin error ni duplicado. Asi, un reintento tras una falla o una doble ejecucion programada no corrompe la tabla. Si hubiera que corregir un valor, se usaria DO UPDATE (upsert).

E3. Formas de programar una carga

La tabla compara tres formas de programar la carga [7][8]. Para este proyecto usaria el Programador de tareas de Windows (o cron en Linux): la carga es un unico script, corre en un solo equipo y no tiene dependencias entre tareas; la bitacora etl_log aporta el monitoreo. Airflow seria mejor si el proyecto crece a varios pipelines dependientes, y pg_cron si la carga fuera solo SQL dentro del servidor.

Aspecto	cron / Programador de tareas	pg_cron	Apache Airflow
Que es	Planificador del sistema operativo que lanza comandos en horarios definidos	Extension de PostgreSQL que ejecuta sentencias SQL programadas dentro del servidor	Plataforma de orquestacion: flujos definidos como DAGs en Python, con dependencias, reintentos y monitoreo web
Cuando conviene	Un script unico en un equipo (caso de run_etl.py)	Tareas puramente SQL: mantenimiento, crear particiones	Muchos pipelines dependientes, reintentos, historial y alertas
Complejidad	Baja	Baja a media (instalar extension; pensada para servidores Linux)	Alta (scheduler, servidor web y base de metadatos)
Limitaciones	Sin dependencias, reintentos ni panel	Solo SQL; no ejecuta Python	Excesivo para un pipeline pequeno

E4. Bitacora del ETL y trazabilidad

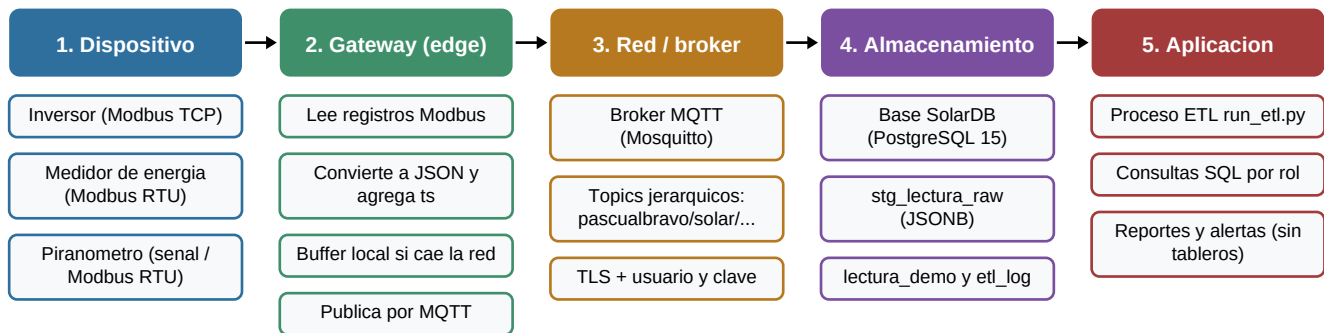
La bitacora debe registrar: identificador de la ejecucion, nombre del proceso, inicio y fin, archivo o fuente procesada, filas leidas, cargadas, duplicadas y rechazadas, estado (EN_CURSO, OK, ERROR) y mensaje de error. Con ellos se detectan cargas vacias, picos de rechazos o fallas. Tambien es una herramienta de gobernanza porque da trazabilidad o data lineage [1]: permite saber de que archivo y en que momento llego cada dato a lectura_demo, que proceso lo cargo y cuantos registros se descartaron. Esto soporta auditorias, la explicacion de cifras ante el owner, la reproduccion de una carga y la responsabilidad sobre la calidad del dato.

```
CREATE TABLE IF NOT EXISTS etl_log (
  id BIGSERIAL PRIMARY KEY, proceso TEXT NOT NULL DEFAULT 'carga_lecturas',
  inicio TIMESTAMPTZ NOT NULL DEFAULT now(), fin TIMESTAMPTZ, archivo TEXT,
  filas_leidas INT, filas_cargadas INT, filas_duplicadas INT, filas_rechazadas INT,
  estado TEXT NOT NULL DEFAULT 'EN_CURSO' CHECK (estado IN ('EN_CURSO','OK','ERROR')),
  error TEXT);
```

Bloque 3. Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

La arquitectura IoT se organiza en capas [4]. Dispositivo: equipos que miden; en la planta son el inversor, el medidor de energia y el piranometro. Gateway (edge): equipo cercano a los dispositivos que lee sus registros por Modbus, normaliza, agrega la marca de tiempo, guarda en buffer si se cae la red y publica los mensajes. Red o broker: transporta los mensajes; en SolarDB, un broker MQTT recibe lo publicado y lo entrega a los suscriptores. Almacenamiento: la base SolarDB en PostgreSQL, con staging JSONB y tablas relacionales. Aplicacion: consultas, reportes y alertas que usan los datos (sin tableros en este trabajo). El diagrama ubica cada elemento.



Flujo: Modbus (campo) -> MQTT (publish/subscribe) -> ETL -> INSERT en PostgreSQL -> consultas

12. MQTT y jerarquia de topics

MQTT [4] es un protocolo liviano de mensajería publish/subscribe: los clientes no se hablan entre sí; publican mensajes en un topic y un broker los reenvía a quienes se suscribieron. Un topic es una ruta jerárquica separada por /, con comodines + (un nivel) y # (varios niveles). QoS 0: como máximo una vez, sin confirmación; QoS 1: al menos una vez, puede haber duplicados; QoS 2: exactamente una vez, con más intercambio de mensajes. Para telemetría recomiendo QoS 1 junto con carga idempotente. Jerarquía propuesta: pascualbravo/solar/<sitio>/<dispositivo>/<variable>, por ejemplo pascualbravo/solar/sede1/inv01/p_ac, .../med01/energia_kwh y .../pir01/irradiancia. El ETL se suscribe a pascualbravo/solar/sede1/#.

13. Modbus, MQTT y OPC UA

La tabla compara los tres valores que admite dispositivo.protocolo [4][10][11]. En la planta, el gateway puede leer el inversor por Modbus y publicarlo por MQTT hacia SolarDB.

	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo de comunicacion	Maestro-esclavo (RTU, serial RS-485) o cliente-servidor (TCP); el maestro consulta registros por polling	Publish/subscribe mediante un broker	Cliente-servidor con suscripciones; también admite pub/sub
Uso tipico	Leer registros de inversores, medidores y PLC en campo	Enviar telemetría desde gateways hacia la nube o una base	Interoperabilidad industrial (SCADA, MES) con modelo de información
Formato de los datos	Registros de 16 bits y bits (coils), sin metadatos; se necesita el mapa de registros	Payload libre (JSON, binario); el protocolo no impone estructura	Nodos tipados con nombre, unidad y descripción
Seguridad	La especificación básica no cifra ni autentica; se usa red aislada o VPN	TLS, usuario y clave o certificados, ACL por topic	Autenticación, firma y cifrado integrados con certificados X.509

14. Payload JSON del inversor y consultas JSONB

El inversor publica cada 5 minutos un mensaje con nueve campos; las unidades van como sufijo en el nombre (kw, v, a, c, kwh, hz). Las consultas usan los operadores nativos de JSONB: -> y ->> para extraer atributos, ? para verificar que exista una clave y @> para buscar por contención.

```
{"device_id": 1, "ts": "2026-10-05T12:00:00-05:00", "p_ac_kw": 3.215,
"dc": {"v_v": 612.4, "i_a": 5.3}, "temp_inv_c": 41.8,
"e_dia_kwh": 12.64, "f_hz": 60.01, "estado": "RUN"}
```

```
-- Consulta 1: -> y ->> (potencia y tension DC del inversor 1)
SELECT payload->>'ts' AS ts, (payload->>'p_ac_kw')::numeric AS p_ac_kw,
      (payload->'dc'->>'v_v')::numeric AS v_dc_v
FROM stg_lectura_raw WHERE (payload->>'device_id')::int = 1;

-- Consulta 2: ? y @> (mensajes con clave de alarma o en estado FAULT)
SELECT payload FROM stg_lectura_raw
WHERE payload ? 'alarma' OR payload @> '{"estado": "FAULT"}';
```

15. Volumen de datos, particionamiento y BRIN

Con 10 dispositivos, 5 variables y una lectura por minuto, si se guarda una fila por variable se generan $10 \times 5 = 50$ filas por minuto, 3.000 por hora, 72.000 por día y 2.160.000 por mes de 30 días (unos 26 millones al año). El particionamiento por rango de ts divide lectura en tablas mensuales: las consultas por fecha leen solo las particiones necesarias, el mantenimiento es más liviano y los datos viejos se archivan con DROP PARTITION [3]. El índice BRIN guarda el mínimo y el máximo de ts por bloques de páginas; como los datos llegan en orden cronológico, es muy pequeño frente a un B-tree y suficiente para filtrar por rangos. TimescaleDB [9] es una extensión que automatiza este particionamiento (hypertables) y agrega compresión.

Bloque 4. Control de versiones

V1. Git, GitHub y manejo de credenciales

Git es un sistema de control de versiones distribuido que funciona localmente y registra la historia del código; GitHub es un servicio en la nube que aloja repositorios Git y agrega colaboración (issues, pull requests, permisos) [5][6]. Repository: carpeta del proyecto con su historial. Commit: instantánea con mensaje de un conjunto de cambios. Branch: línea de desarrollo independiente para trabajar sin afectar la principal. Pull request: solicitud para revisar e integrar los cambios de una rama en otra. .gitignore: lista de lo que Git no debe rastrear. Nunca se suben credenciales porque el historial es permanente, el repositorio es público y existen bots que buscan secretos; se evita con variables de entorno, un .env ignorado, un .env.example sin valores reales y rotando la clave si se filtra.

Parte B. Práctica guiada: primer pipeline de ingesta

Entorno: PostgreSQL 15 o superior, Python y psycopg2. El código completo está en el repositorio (carpetas etl/ y sql/); aquí se muestran solo los fragmentos clave y la evidencia.

Paso 1. Simulador IoT

Se modificó etl/simulador.py para generar lecturas de dos dispositivos (device_id 1 y 2) y se agregó el campo voltaje_dc (V), la tensión DC del inversor. Cada ejecución produce 288 mensajes JSON (144 por dispositivo, uno cada 5 minutos entre las 6:00 y las 17:55) en data/lecturas.jsonl, con semilla fija para que el archivo sea reproducible.

```
for i in range(144):
    for device_id in [1, 2]:
        # dos dispositivos
        msg = {"device_id": device_id, "ts": ..., "p_ac": ..., "irradiancia": ...,
              "temp_modulo": ..., "voltaje_dc": round(random.uniform(300, 800), 1)} # campo nuevo
```

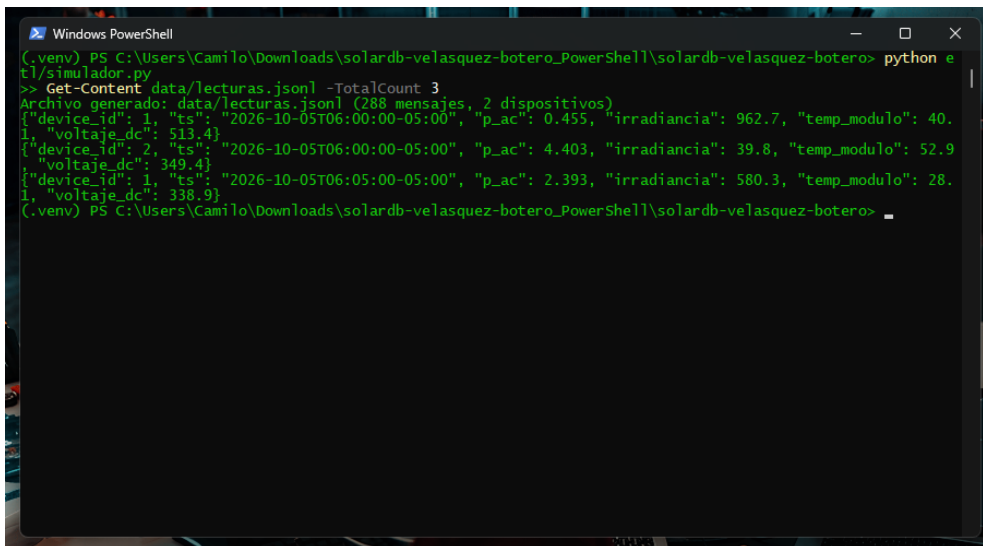


Figura: Ejecución de python etl/simulador.py y primeras líneas de data/lecturas.jsonl

Paso 2. Staging y tabla destino

Las tablas se crean con sql/01_tablas.sql. El archivo se carga a stg_lectura_raw con un script en Python (psycopg2) y no con \copy: el flujo debe correr con un solo comando, el script valida que cada línea sea JSON antes de insertarla y no depende de psql. \copy es más rápido con millones de filas, pero en modo texto interpreta las barras invertidas y no valida el contenido. Resultado esperado: 288 filas en staging.

```
python etl/run_etl.py --paso staging
psql -U postgres -d solardb -c "SELECT COUNT(*) FROM stg_lectura_raw;"
```

```
Windows PowerShell
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> python etl/run_etl.py --paso staging
>> psql -U postgres -d solardb -c "\dt"
>> psql -U postgres -d solardb -c "SELECT COUNT(*) FROM stg_lectura_raw;"
stg_lectura_raw: 288 filas

List of tables
Schema | Name | Type | Owner
-----|-----|-----|-----
public | etl_log | table | postgres
public | lectura_demo | table | postgres
public | stg_lectura_raw | table | postgres
(3 rows)

count
-----
288
(1 row)

(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> _
```

Figura: Tablas creadas (\dt) y conteo de stg_lectura_raw

Paso 3. Transformacion y carga

La sentencia INSERT ... SELECT (sql/02_carga.sql) extrae cada atributo con ->>, lo convierte al tipo de lectura_demo con :: y guarda ademas el mensaje completo en payload. ON CONFLICT (dispositivo_id, ts) DO NOTHING omite las lecturas que ya existen. Resultado esperado en la primera ejecucion: INSERT 0 288.

```
INSERT INTO lectura_demo (dispositivo_id, ts, p_ac, irradiancia, temp_modulo, payload)
SELECT (payload->>'device_id')::int, (payload->>'ts')::timestampz,
      (payload->>'p_ac')::numeric, (payload->>'irradiancia')::numeric,
      (payload->>'temp_modulo')::numeric, payload
FROM stg_lectura_raw ORDER BY id
ON CONFLICT (dispositivo_id, ts) DO NOTHING;
```

```
Windows PowerShell
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> psql -U postgres -d solardb -f sql/02_carga.sql
>> psql -U postgres -d solardb -c "SELECT dispositivo_id, ts, p_ac, irradiancia, temp_modulo FROM lectura_demo ORDER BY ts LIMIT 5;"
INSERT 0 288
dispositivo_id | ts | p_ac | irradiancia | temp_modulo
-----|-----|-----|-----|-----
2 | 2026-10-05 06:00:00-05 | 4.403 | 39.8 | 52.9
1 | 2026-10-05 06:00:00-05 | 0.455 | 962.7 | 40.1
1 | 2026-10-05 06:05:00-05 | 2.393 | 580.3 | 28.1
2 | 2026-10-05 06:05:00-05 | 0.490 | 103.9 | 31.9
1 | 2026-10-05 06:10:00-05 | 0.507 | 51.7 | 58.9
(5 rows)

(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> _
```

Figura: Ejecucion de sql/02_carga.sql (INSERT 0 288) y SELECT de muestra

Paso 4. Prueba de idempotencia

Se ejecuto el flujo dos veces con el mismo archivo. Despues de la primera ejecucion COUNT(*) es 288; despues de la segunda sigue en 288, porque las 288 lecturas ya existian y ON CONFLICT DO NOTHING las omitio. Esto demuestra idempotencia: repetir la carga no duplica datos.

```
python etl/run_etl.py
psql -U postgres -d solardb -c "SELECT COUNT(*) FROM lectura_demo;"
python etl/run_etl.py
psql -U postgres -d solardb -c "SELECT COUNT(*) FROM lectura_demo;"
```

```

Windows PowerShell
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> python etl/run_etl.py
>> psql -U postgres -d solardb -c "SELECT COUNT(*) FROM lectura_demo;"
>> python etl/run_etl.py
>> psql -U postgres -d solardb -c "SELECT COUNT(*) FROM lectura_demo;"
ETL OK: leidas=288 cargadas=288 duplicadas=0 rechazadas=0
SELECT COUNT(*) FROM lectura_demo -> 288
count
-----
288
(1 row)

ETL OK: leidas=288 cargadas=0 duplicadas=288 rechazadas=0
SELECT COUNT(*) FROM lectura_demo -> 288
count
-----
288
(1 row)

(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero>

```

Figura: SELECT COUNT(*) FROM lectura_demo despues de cada una de las dos ejecuciones

Paso 5. Bitacora y automatizacion

etl_log registra cada ejecucion (inicio, fin, filas leidas, cargadas, duplicadas, rechazadas, estado y error). Todo el flujo se lanza con un unico comando: python etl/run_etl.py. En la segunda ejecucion la bitacora muestra 0 filas cargadas y 288 duplicadas. La programacion cada hora se deja escrita, sin implementarla.

```

# cron (cada hora, minuto 0)
0 * * * * cd /ruta/solardb-velasquez-botero && python etl/run_etl.py >> logs/etl.log 2>&1

:: Programador de tareas de Windows (equivalente)
schtasks /Create /SC HOURLY /TN "SolarDB_ETL" /TR "cmd /c cd /d C:\ruta\solardb-velasquez-botero && python etl\run_etl.py"

```

```

Windows PowerShell
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> python etl/run_etl.py
>> psql -U postgres -d solardb -c "SELECT id, estado, filas_leidas, filas_cargadas, filas_duplicadas, filas_rechazadas FROM etl_log ORDER BY id;"
ETL OK: leidas=288 cargadas=0 duplicadas=288 rechazadas=0
SELECT COUNT(*) FROM lectura_demo -> 288
id | estado | filas_leidas | filas_cargadas | filas_duplicadas | filas_rechazadas
---|---|---|---|---|---
1  | OK     | 288          | 288            | 0                | 0
2  | OK     | 288          | 0              | 288              | 0
3  | OK     | 288          | 0              | 288              | 0
(3 rows)

(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero>

```

Figura: Ejecucion de run_etl.py y SELECT * FROM etl_log

Paso 6. Gobernanza aplicada

El script sql/03_rols.sql crea el rol solar_lector con GRANT SELECT sobre lectura_demo. Al conectarse con ese rol, el SELECT funciona y el INSERT es rechazado con permission denied, lo que confirma el principio de minimo privilegio.

```
psql -U solar_lector -d solardb -f sql/04_prueba_rol.sql -- SELECT ok, INSERT rechazado
```



```
Windows PowerShell
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero> Remove-Item Env:PGPASSWORD
>> psql -U solar_lector -d solardb -f sql/04_prueba_rol.sql
Password for user solar_lector:

  usuario
-----
solar_lector
(1 row)

  filas
-----
    288
(1 row)

psql:sql/04_prueba_rol.sql:9: ERROR: permission denied for table lectura_demo
(.venv) PS C:\Users\Camilo\Downloads\solardb-velasquez-botero_PowerShell\solardb-velasquez-botero>
```

Figura: Rol solar_lector: SELECT exitoso e INSERT rechazado

Parte C. Repositorio en GitHub

Enlace al repositorio: <https://github.com/CamiloVelasquezBotero/solardb-velasquez-botero>

El repositorio es publico y se llama solardb-velasquez-botero. Contiene README.md (nombre completo, curso, grupo, descripcion, pasos para reproducir y tabla de quien hizo que), .gitignore, y las carpetas docs/, data/, etl/ y sql/. Las credenciales se manejan con variables de entorno y un archivo .env excluido del repositorio. El trabajo fue realizado en pareja por Camilo Velasquez Botero y Deivy Estiben Gomez Rivera. Camilo creo el repositorio y agrego a Deivy como colaborador; ambos integrantes aportaron commits con mensajes descriptivos, como se ve en las capturas.

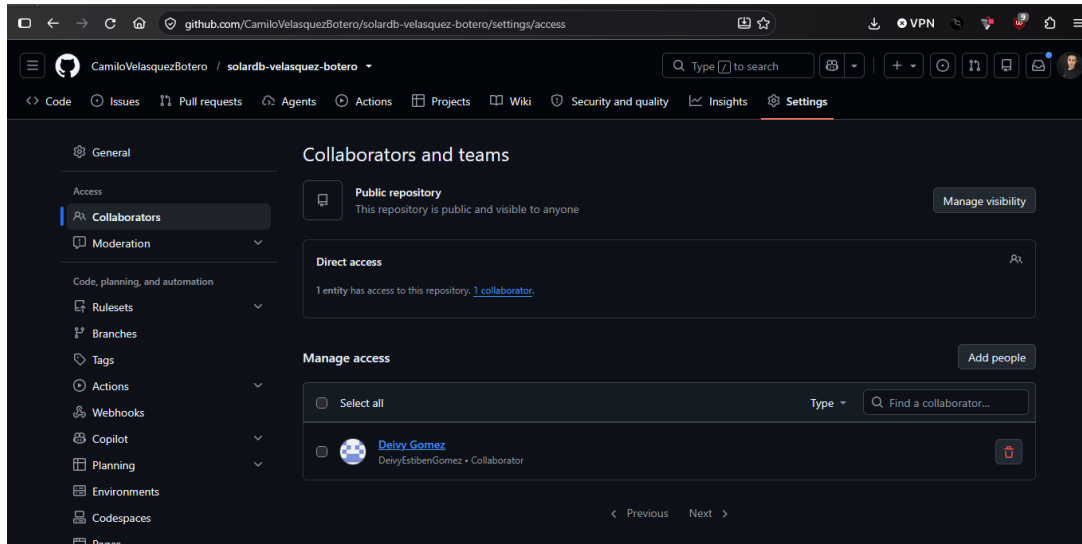


Figura: Settings > Collaborators del repositorio (ambos integrantes)

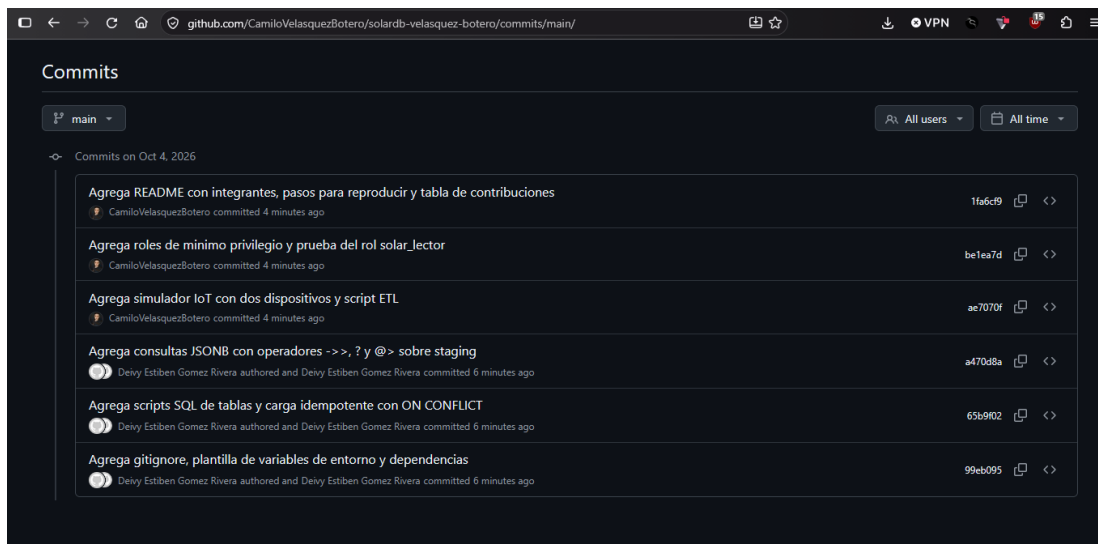


Figura: Historial de commits del repositorio (autores: ambos integrantes)

Referencias

- [1] DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2.a ed.). Technics Publications. <https://www.dama.org>
- [2] Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012*, por la cual se dictan disposiciones generales para la proteccion de datos personales. https://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html
- [3] PostgreSQL Global Development Group. *PostgreSQL 15 Documentation* (INSERT ... ON CONFLICT, CREATE ROLE, GRANT, JSON functions and operators, BRIN indexes, table partitioning). <https://www.postgresql.org/docs/15/>
- [4] OASIS. (2019). *MQTT Version 5.0*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [5] Chacon, S. y Straub, B. (2014). *Pro Git* (2.a ed.). Apress. <https://git-scm.com/book/es/v2>

- [6] GitHub. *GitHub Docs*. <https://docs.github.com>
- [7] The Apache Software Foundation. *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [8] Citus Data. *pg_cron: job scheduler for PostgreSQL*. https://github.com/citusdata/pg_cron
- [9] Timescale. *TimescaleDB Documentation*. <https://docs.timescale.com/>
- [10] Modbus Organization. *Modbus Application Protocol Specification V1.1b3*. <https://modbus.org/specs.php>
- [11] OPC Foundation. *OPC Unified Architecture (OPC UA)*. <https://opcfoundation.org/about/opc-technologies/opc-ua/>
- [12] Saltzer, J. y Schroeder, M. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308.
- [13] Superintendencia de Industria y Comercio (SIC). *Proteccion de datos personales*. <https://www.sic.gov.co>
- [14] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [15] Kimball, R. y Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley.